

Database

CRM/ERP Platform Documentation

Database

Overview

- **Engine:** MySQL 8.0
 - **ORM:** Prisma 5
 - **Schema:** `backend/prisma/schema.prisma`
 - **Migrations:** `backend/prisma/migrations/`
 - **Models:** 90 total
-

Running Migrations

```
cd backend

# Apply pending migrations (production / CI)
npx prisma migrate deploy

# Create + apply a new migration (development)
npx prisma migrate dev --name description_of_change

# Regenerate Prisma client after schema changes
npx prisma generate

# Open visual browser for the database
npx prisma studio
```

Schema Overview

Core: Organization & Users

Model	Table	Key Fields
Organization	organizations	id , slug (unique), name , settings (JSON)
User	users	id , email (unique), role (enum), organizationId
Department	departments	id , slug+orgId (unique), permissions (JSON)

User roles (enum):

SUPER_ADMIN → ADMIN → MANAGER → USER → VIEWER

User statuses: ACTIVE , INACTIVE , SUSPENDED , LOCKED , PENDING

Dynamic Data: Modules & Fields

Model	Table	Key Fields
DynamicModule	dynamic_modules	id , slug+orgId (unique), settings (JSON)
Field	fields	id , name , type (enum), moduleId , options
FieldOption	field_options	id , label , value , color , fieldId
FieldRule	field_rules	id , conditions (JSON), actions (JSON)
Record	records	id , moduleId , data (JSON), isDeleted
Comment	comments	id , recordId , userId , content
File	files	id , recordId , url , mimeType

Field types (enum):

TEXT, TEXTAREA, RICH_TEXT, NUMBER, DECIMAL, CURRENCY, BOOLEAN, CHECKBOX
RADIO, DROPDOWN, MULTI_SELECT, DATE, DATETIME, EMAIL, PHONE, URL

FILE, IMAGE, SIGNATURE, USER_SELECT, TAGS, FORMULA, LOOKUP
AUTO_NUMBER, STATUS, RATING, PROGRESS, COLOR_PICKER
GLOBAL_RELATION, INLINE_SUBFORM, MIRROR

Records store all field values in `data: Json` — a JSON blob keyed by field name. There is no separate column per field.

Relationships & Views

Model	Table	Key Fields
Relationship	relationships	type (enum), fromModuleId , toModuleId
View	views	type (enum), config (JSON), filters (JSON), columns (JSON)

View types: TABLE , KANBAN , CALENDAR , GALLERY , FORM , TIMELINE

Relation types: ONE_TO_ONE , ONE_TO_MANY , MANY_TO_ONE , MANY_TO_MANY

Workflows & Blueprints

Model	Table	Key Fields
Workflow	workflows	trigger (enum), conditions (JSON), isActive
WorkflowAction	workflow_actions	type (string), config (JSON), order
WorkflowExecution	workflow_executions	status , input / output (JSON), error
Blueprint	blueprints	phases (JSON), transitions (JSON), fieldLocks (JSON)
BlueprintTask	blueprint_tasks	status , assignedToId , fromStage , toStage

Workflow triggers: RECORD_CREATED , RECORD_UPDATED , RECORD_DELETED , FIELD_CHANGED , SCHEDULED , MANUAL , WEBHOOK , FORM_SUBMITTED

Forms

Model	Table	Key Fields
Form	forms	token (unique), settings (JSON), moduleId
FormSection	form_sections	label, order, formId
FormField	form_fields	fieldId, conditionalLogic (JSON), customLabel
FormPermission	form_permissions	role, canSubmit, canView
FormSubmission	form_submissions	data (JSON), ticketNumber, ipAddress
FormFolder	form_folders	name, color, organizationId

`Form.settings` is a freeform JSON that stores extended config without migrations:

```
{
  "documents": { "enabled": true, "fieldIds": ["..."] },
  "ticketing": { "enabled": true, "message": "..." }
}
```

Permissions

Model	Table	Scope
Permission	permissions	Per role per module
UserPermissionOverride	user_permission_overrides	Per user per module

Permission resolution order: SUPER_ADMIN bypass → role permission → department override → user override.

Portal

Model	Table	Notes
PortalUser	portal_users	Separate from <code>User</code> ; external customers

Model	Table	Notes
PortalPage	portal_pages	Configurable pages with blocks
PortalSection	portal_sections	Sections within pages
PortalField	portal_fields	Fields shown in portal
PortalSettings	portal_settings	Password policy per org
PortalDocument	portal_documents	Files uploaded by portal users

Analytics & Dashboards

Model	Table	Key Fields
Dashboard	dashboards	sharedRoles , sharedDepts , sharedUsers (JSON arrays)
DashboardWidget	dashboard_widgets	type (enum), config (JSON), position (JSON)
AnalyticsView	analytics_views	config (JSON)
AnalyticsTarget	analytics_targets	targetValue , aggregation , period

Widget types: KPI_CARD , BAR_CHART , PIE_CHART , LINE_CHART , AREA_CHART , DONUT_CHART , TABLE , FUNNEL

Other Models

Domain	Models
Global Lists	Globallist , GloballistItem (hierarchical, with cross-list linking)
Workspace	WorkspaceTask , WorkspaceSubtask , WorkspaceChecklist , WorkspaceNote
Messaging	Conversation , ConversationParticipant , DirectMessage
Calendar	UserCalendarConnection , CalendarEventMapping
Tracker	Tracker , TrackerCriteria , TrackerSession , TrackerScore , TrackerBand

Domain	Models
Requests	RequestType , Request , RequestBlueprint , RequestBlueprintStage , RequestInstance
Gallery & Pubs	GalleryFile , Publication , PublicationAttachment , PublicationEngagement
Email	EmailTemplate , EmailLog , SmtSettings
Routing	RecordRoutingConfig
Security	AuditLog , ApiKey , Notification

Migration History

Migration	Date	Description
20260523061724_init	2026-05-23	Initial schema
20260523073937_...	2026-05-23	Global lists, forms, permissions, global relation
20260625000000_add_email_tables	2026-06-25	EmailTemplate, EmailLog
20260629000000_add_routing_workflow_email_design	2026-06-29	Routing, workflow email design field
20260630000000_add_team_roles	2026-06-30	Team roles
20260630000001_replace_team_roles_with_global_list	2026-06-30	Roles via global list
20260630000002_add_blueprint_request_tracking	2026-06-30	Blueprint + request fields
20260701000000_add_field_rules	2026-07-01	FieldRule model

Migration	Date	Description
20260701000000_extend_blueprint_tasks_queue	2026-07-01	BlueprintTask extensions
20260701000001_purge_soft_deleted_blueprints	2026-07-01	Cleanup migration
20260702000000_add_ticket_number_to_submissions	2026-07-02	ticketNumber on FormSubmission

Soft Deletes

Records use `isDeleted: Boolean @default(false)` + `deletedAt`. The service layer adds `where: { isDeleted: false }` to all queries. Permanently deleted records call `prisma.record.delete()`.

Avoid querying `records` table without the `isDeleted` filter.